Class and instance (static)

4

One **class** object is created for every class (automatically), and various **instance** objects may also be created (from classes / class objects with **constructors**).

Data fields and methods are called **members** of a class. A member can be declared with **static** (it is part of the **class** object, a **class member**), or without static (it is part of any **instance** objects from the class, an **instance member**). the terminology is a bit confusing

Class members (static) are created and usable before the program starts to run (like the main method, Math.PI, Math.sqrt()).

Members of the application class are usually class members (static), support classes usually not.

The program can use **static** / class members and the instance members of instance objects.

These are the solid bits of my models, not the *dashed*

So:

(1) We can use class members in class objects: ClassName.memberName

(2) We can't use instance members in class objects (they don't really exist!)

(3) We can use instance members in instance objects: variableName.memberName

(4) We can seem to use class members in instance objects: variableName.memberName but we are really using the member in the underlying class object instead.

See Cases 1 - 4 on the next slide in an example using data fields…

String methods

The class String has (and so String objects have) many useful methods. Assume:

```
String hi = "Hello All!";
```

0123456789 – positions in the string (computer scientists number from 0)

length() returns the length of a string:

hi.length() is 10 "Anthony".length() is 7

literal strings (Lec 4) like this create a normal string object with a "temporary" reference that is not stored in a variable

indexOf() returns the position of a char (character) or string:

hi.indexOf('e') is 1 hi.indexOf('x') is –1 "wibble".indexOf('w') is 0

not found

hi.indexOf("lo") is 3

substring() returns a specified part of a string:

hi.substring(3, 9) is "lo All" "a1#2c".substring(0, 2) is "a1"

from not including

replace() replaces every occurrence of an old char with a new one:

hi.replace('l','X') is "HeXXo AXX!"

charAt() returns the character at the specified position:

hi.charAt(1) is 'e'

ASCII Char values:

0 = 48
9 = 57
A = 65
Z = 90
a = 97
z = 122

Graphics import:

import javax.swing.*;
import java.awt.*;

Random

Random generator = new Random();
generator.nextInt(21) 0-20